

考试科目名称 《计算机图形学》 (A 卷)

2021——2022 学年第 一 学期 教师 孙正兴、张岩 考试方式：闭卷

系（专业） 年级 班级

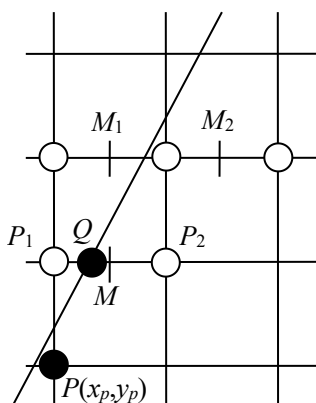
学号 姓名 成绩

题号	一	二	三	四	五
分数					

得分 (本题满分 20 分)

1. 写出 $|k|>1$ 的直线中点画线算法。（要求写清原理、构造的判别式、递推公式、算法伪代码、写清注释，并画图解释）

解析：



构造判别式： $d=F(M)=F(x_p+0.5, y_p+1)=a(x_p+0.5)+b(y_p+1)+c$

- 当 $d<0$ ， M 在 Q 点左侧，取右上方 P_2 为下一个像素；
- 当 $d>0$ ， M 在 Q 点右侧，取上方 P_1 为下一个像素；
- 当 $d=0$ ，选 P_1 或 P_2 均可，约定取 P_1 为下一个像素；

增量计算：

- 若 $d\geq 0$ ，取正上方像素 $P_1(x_p, y_p+1)$ ，要判下一个像素位置，应计算

$$d_1=F(M_1)=F(x_p+0.5, y_p+2)=a(x_p+0.5)+b(y_p+2)+c=d+b$$

增量为 b 。

- 若 $d<0$ ，取右上方像素 $P_2(x_p+1, y_p+1)$ 。要判断再下一像素，则要计算

$$d_2=F(M_2)=F(x_p+1.5, y_p+2)=a(x_p+1.5)+b(y_p+2)+c=d+a+b$$

增量为 $a+b$ 。

- d 的初值 $d_0=F(x_0+0.5, y_0+1)=F(x_0, y_0)+0.5a+b=0.5a+b$

用 $2d$ 代替 d 的算法：

void Midpoint Line (int x0,int y0,int x1, int y1,int color)

{ int a, b, d1, d2, d, x, y;

```

a=y0-y1, b=x1-x0, d=a+2*b;
d1=2*b, d2=2*(a+b);
x=x0, y=y0;
drawpixel(x, y, color);
while (y<y1)
{ if (d<0)          {x++, y++, d+=d2; }
  else              {y++, d+=d1; }
  drawpixel (x, y, color);
} /* while */
} /* mid ointLine */

```

得分		(本题满分 20 分)
----	--	-------------

2. 修改 Cohen-Sutherland 直线裁剪算法，使其成为一个直线“开窗”算法，即指定一个窗口后，窗口内舍弃，窗口外保留。（要求说明对原算法的修改位置，给出具体算法伪代码并给出代码清晰注释）

解析：根据题意，可知只需要对原 Cohen-Sutherland 算法中的两处进行修改即可满足要求。

第一处是判断 C_1 和 C_2 的逻辑乘结果不为 0 时，此时如果该条件满足，表示线段完全在窗外，原算法此处需要将原线段完全舍弃，这里就需要将原线段完全绘制出来即可。

第二处是最后要将原线段中窗口中可见部分绘制出来，此时原算法已经完成对原线段的裁剪，得出来原线段在窗口内的部分，这里只需要改成将原线段去掉窗口以内部分后的线段绘制出来即可。

根据以上分析，修改 Cohen-Sutherland 直线裁剪算法为直线“开窗”算法如下：

double xl, xr, yt, yb; (这里事先给出窗口的位置，四个数值是已知的)，修改的部分用蓝色表示 void Cohen_Sutherland(double x0, y0, x2, y2)

```

{
    int c, c1, c2;
    double x, y;
    //需要将原线段的端点保存起来，以备后面需要确定原线段去除窗口内部分时使用
    double x00=x0, y00=y0, x22=x2, y22=y2;
    makecode(x0, y0, c1); makecode(x2, y2, c2);
    while (c1!=0 || c2!=0)
    {
        if (c1&c2!=0) {
            showline(x00, y00, x22, y22); //显示原线段，能走到这说明原线段都在窗口外
            return;
        }
        c=c1; if (c==0) c=c2;
        if (c&1==1) {y=y0+(y2-y0)*(x1-x0)/(x2-x0); x=x1;}
        else if (c&2==2) {y=y0+(y2-y0)*(xr-x0)/(x2-x0); x=xr;}
        else if (c&4==4) {x=x0+(x2-x0)*(yb-y0)/(y2-y0); y=yb;}
    }
}

```

```

Else          if          (c&8==8)
{x=x0+(x2-x0)*(yt-y0)/(y2-y0); y=yt;} if
(c==c1)
{x0=x;          y0=y;
makecode(x, y, c1);}
else
{x2=x; y2=y; makecode(x, y, c2);}
}

```

//因为原算法的线段分割保证了端点的顺序性，所以采用如下的方法可确定

原线段在窗口外的部分

```

if (x00!=x0 || y00!=y0)
showline(x00,y00,x0,y0); if
(x2!=x22 || y2!=y22)
showline(x2,y2,x22,y22);

```

另一种方法是在分割线段的同时绘制窗口外的线段，该方法无需记录初始点坐标。

double xl, xr, yt, yb; (这里事先给出窗口的位置，四个数值是已知的)，修改的部分用蓝色表示

```

void Cohen_Sutherland(double x0, y0, x2, y2)
{
    int c, c1, c2; double x, y;
    makecode(x0,          y0,c1);
    makecode(x2, y2, c2); while
    (c1!=0 || c2!=0)
    {
        if (c1&c2!=0) {
//显示线段，此时绘制的是分割完后，完全在窗口外

同侧的线段 showline(x0, y0, x2, y2);

return;
        }
        c=c1; if (c==0) c=c2;
        if
                                (c&1==1)

```

```

    {y=y0+(y2-y0)*(x1-x0)/(x2-x0);    x=x1;}
else
    if
        (c&2==2)
    {y=y0+(y2-y0)*(xr-x0)/(x2-x0); x=xr;} else
if
        (c&4==4)
    {x=x0+(x2-x0)*(yb-y0)/(y2-y0); y=yb;} else
if (c&8==8) {x=x0+(x2-x0)*(yt-y0)/(y2-y0);
y=yt;} if (c==c1)
{showline(x0,y0,x,y);//绘制被分割抛弃的线段
x0=x;          y0=y;
makecode(x, y, c1);}

else
{showline(x,y,x2,y2);//绘制被分割抛弃的线段
x2=x; y2=y; makecode(x, y, c2);}

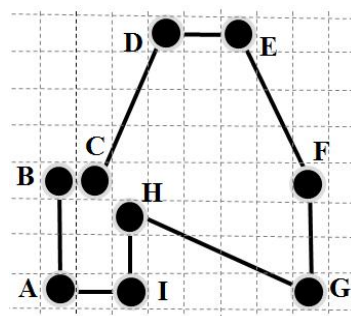
}

}

```

得分		(本题满分 20 分)
----	--	-------------

3. 如下图给出了多边形区域 ABCDEFGHIA 的定义，各顶点坐标分别为：A(0,0)、B(0,8)、C(5,8)、D(11,18)、E(18,18)、F(23,8)、G(23,0)、H(6,7)和 I(6,0)，需采取与顶点坐标对应的屏幕像素位置的方式对多边形区域进行填充。试回答下列问题：(1) 若采用多边形扫描转换方法对图中多边形区域进行均匀着色填充，那么，填充过程需包含哪些主要步骤？正确填充区域的关键环节是什么？(2) 什么是直线的连贯性？如何运用直线连贯性原理并采用整数增量运算方法计算出线段 CD 与扫描线的交点像素位置？(3) 若分别去除图中的线段 AB、BC、DE、FG 和 AI，那么，能否有效填充该多边形区域？(4) 若将多边形区域转换成像素点阵表示，那么，如何定义内部像素和边界像素的连通性才能保证采用递归区域填充方法正确填充该区域？



多边形的定义方式

解析：

(1). 基本算法过程：（扫描线与多边形边界求交→交点处理→）（交点成对并存储→确定各扫描线上的内部像素点位置）→（设置内部像素颜色）。

或：扫描转换填充：求出各扫描线与边界的交点，堆栈中存储各扫描线上最左侧的交点，采用“先进后出”的方式按交点所在扫描线逐条设置扫描线上的像素颜色。

关键环节：扫描线与边界求交处理，要点：

1) 水平边 BC、DE 和 AI 不参与求交，直接作为内部点；

2) 多边形顶点求交处理：F 计为扫描线上的 1 个交点，其余顶点计为扫描线上的 2 个交点。

(2). 直线的连贯性：直线跨越扫描线时其斜率保持不变。

增量运算：边界与当前扫描线的交点是在该边与前一条扫描线交点基础上增加该边斜率的倒数。

A 为屏幕像素坐标原点，C(5,8)、D(11,18)， $\nabla x=6$ ， $\nabla y=10$ ， $\nabla x/\nabla y=3/5$

计数器每步增加 $2\nabla x$ 并与 ∇y 比较→X 方向增加 1，计数器减去 $2\nabla y$

交点位置：(5,8)、(6,9)、(7,11)、(8,12)、(9,14)、(10,16)、(11,18)

或：计数器每步增加 ∇x 并与 ∇y 比较→X 方向增加 1，计数器减去 ∇y

交点位置：(5,8)、(6,10)、(7,12)、(8,14)、(9,15)、(10,17)、(11,18)

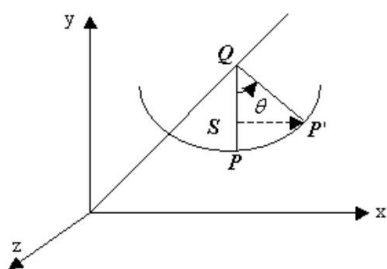
(3). 水平线段 BC、DE 和 AI 不影响填充结果（2 分）；垂直线段 AB 和 FG 会影响填充结果。

(4). 递归填充：任意选择种子点，从种子点开始，按内部像素连通性递归地逐个像素地检测并扩散到整个区域。→区域内部像素定义为八连通，区域边界像素定义为四连通；

得分		（本题满分 20 分）
----	--	-------------

4. 试用下图提示的方法，求出绕通过原点的任意直线旋转 θ 角的变换矩阵。这里沿过原点任意直线的单位方向向量 (n_1, n_2, n_3) 是已知的，图中任意点 P 变换后成为点 P' ，有关系

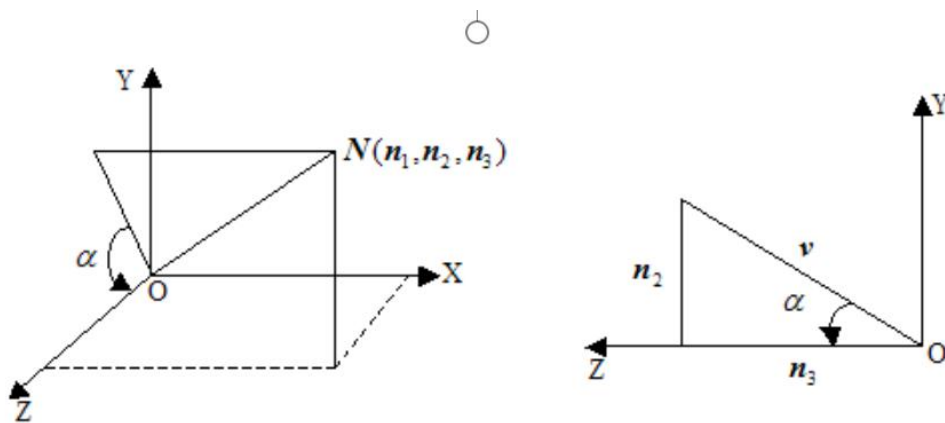
$\overrightarrow{OP}^* = \overrightarrow{OQ} + \overrightarrow{QS} + \overrightarrow{SP}^*$ 。（要求写清旋转变换步骤并画图解释，给出旋转矩阵推导过程）



绕过原点任意一条直线旋转

解析： $R(\theta)$ 是实现以过坐标原点的任意直线为旋转轴做旋转变换的变换矩阵。按所设，旋转轴方向由单位向量 $[n_1 \quad n_2 \quad n_3]$ 确定。这个以过坐标原点的任意直线为旋转轴的旋转变换可分为五步实现：

- (1) 做绕 x 轴旋转 α 角的变换 $R_x(\alpha)$ ，使旋转轴落在 $y = 0$ 平面上。
- (2) 做绕 y 轴旋转 β 角的变换 $R_y(\beta)$ ，使旋转轴与 z 轴重合。
- (3) 做绕 z 轴旋转 θ 角的旋转变换。
- (4) 做第 2 步的逆变换，即做旋转变换 $R_y(-\beta)$ 。
- (5) 做第 1 步的逆变换，即做旋转变换 $R_x(-\alpha)$ 。



绕 X 轴旋转 α 角（右面是在 $X=0$ 平面上的投影）

先看第一步，如上图所示，图中 O 是坐标原点，点 N 的坐标是 (n_1, n_2, n_3) ，因此 ON 是被作为旋转轴的任意直线。记 $v = \sqrt{n_2^2 + n_3^2}$ ，就有：

$$\cos \alpha = \frac{n_3}{v}, \quad \sin \alpha = \frac{n_2}{v}$$

因此可得：

$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \frac{n_3}{v} & \frac{n_2}{v} & 0 \\ 0 & -\frac{n_2}{v} & \frac{n_3}{v} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

此变换将点 N 变换成 N' ，用齐次坐标记法有：

$$N' = \begin{bmatrix} n_1 & n_2 & n_3 & 1 \end{bmatrix} R_x(\alpha) = \begin{bmatrix} n_1 & 0 & v & 1 \end{bmatrix}$$

可见旋转轴已经落在 $y = 0$ 平面上了。

第二步，如图 3.16 所示，记 $d = \sqrt{n_1^2 + n_2^2 + n_3^2}$ ，这里要注意到是面向 y 轴，图中的 β 角是沿顺时针方向给出的，按照对旋转正方向的约定，这是旋转的负方向。再因为 $d = 1$ ，所以有：

$$\cos \beta = \frac{v}{d} = v, \quad \sin \beta = -\frac{n_1}{d} = -n_1$$

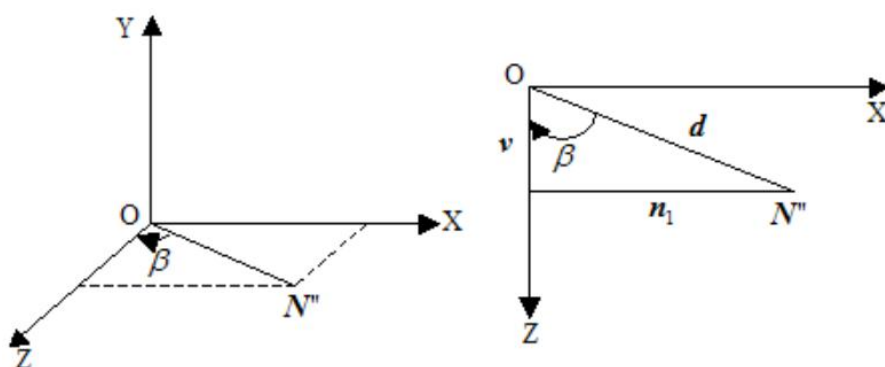
因此所做变换的矩阵是：

$$R_y(\beta) = \begin{bmatrix} v & 0 & n_1 & 0 \\ 0 & 1 & 0 & 0 \\ -n_1 & 0 & v & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

此变换将点 N' 变换成 N'' ，用齐次坐标记法有：

$$N'' = \begin{bmatrix} n_1 & 0 & v & 1 \end{bmatrix} R_y(\beta) = \begin{bmatrix} 0 & 0 & 1 & 1 \end{bmatrix}$$

可见旋转轴已经落到 z 轴上。



绕 Y 轴旋转 β 角（右面是在 $Y=0$ 平面上的投影）

第三步，绕 z 轴旋转 θ 角，变换矩阵 $R_z(\theta)$ 是：

$$R_z(\theta) = \begin{bmatrix} \cos \theta & \sin \theta & 0 & 0 \\ -\sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

第四步，变换矩阵 $R_y(-\beta)$ 是：

$$R_y(-\beta) = \begin{bmatrix} v & 0 & -n_1 & 0 \\ 0 & 1 & 0 & 0 \\ n_1 & 0 & v & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

第五步，变换矩阵 $R_x(-\alpha)$ 是：

$$R_x(-\alpha) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \frac{n_3}{v} & -\frac{n_2}{v} & 0 \\ 0 & \frac{n_2}{v} & \frac{n_3}{v} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

至此可知实现以过坐标原点任意直线为旋转轴的旋转变换的变换矩阵 $R(\theta)$ 是：

$$R(\theta) = R_x(\alpha) \bullet R_y(\beta) \bullet R_z(\theta) \bullet R_y(-\beta) \bullet R_x(-\alpha)$$

$$= \begin{bmatrix} n_1^2 + (1 - n_1^2) \cos \theta & n_1 n_2 (1 - \cos \theta) + n_3 \sin \theta & n_1 n_3 (1 - \cos \theta) - n_2 \sin \theta & 0 \\ n_1 n_2 (1 - \cos \theta) - n_3 \sin \theta & n_2^2 + (1 - n_2^2) \cos \theta & n_2 n_3 (1 - \cos \theta) + n_1 \sin \theta & 0 \\ n_1 n_3 (1 - \cos \theta) + n_2 \sin \theta & n_2 n_3 (1 - \cos \theta) - n_1 \sin \theta & n_3^2 + (1 - n_3^2) \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

得分 （本题满分 20 分）

5. 设用 P_0, P_1, P'_0, P'_1 确定了一段 Hermite 形式的参数三次多项式曲线，用 Q_0, Q_1, Q'_0, Q'_1 又确定了一段。

1) 问两段曲线连续的条件是什么？

2) 二阶导数和二阶导数连续的条件是什么？

3) 由此讨论两段 Hermite 形式的参数三次曲线怎样才能光滑地拼接起来？

$$P(u) = \mathbf{P}_0 \cdot (2u^3 - 3u^2 + 1) + \mathbf{P}_1 \cdot (-2u^3 + 3u^2) + \mathbf{P}'_0 \cdot (u^3 - 2u^2 + u) + \mathbf{P}'_1 \cdot (u^3 - u^2)$$

解析:

$$= \begin{pmatrix} u^3 & u^2 & u & 1 \end{pmatrix} \begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} \mathbf{P}_0 \\ \mathbf{P}_1 \\ \mathbf{P}'_0 \\ \mathbf{P}'_1 \end{bmatrix}$$

两段曲线连续的条件是 $P(1) = Q(0)$ ，即 $\mathbf{P}_1 = \mathbf{Q}_0$

$$P'(u) = \mathbf{P}_0 \cdot (6u^2 - 6u) + \mathbf{P}_1 \cdot (-6u^2 + 6u) + \mathbf{P}'_0 \cdot (3u^2 - 4u + 1) + \mathbf{P}'_1 \cdot (3u^2 - 2u)$$

$$= \begin{pmatrix} u^2 & u & 1 \end{pmatrix} \begin{bmatrix} 6 & -6 & 3 & 3 \\ -6 & 6 & -4 & -2 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} \mathbf{P}_0 \\ \mathbf{P}_1 \\ \mathbf{P}'_0 \\ \mathbf{P}'_1 \end{bmatrix}$$

同理可得 $Q'(u)$

一阶导数连续的条件是 $P'(1) = a \cdot Q'(0)$ ，(a 是比例系数)，即 $\mathbf{P}'_1 = a\mathbf{Q}'_0$ 。当 $a=1$ 时， $\mathbf{P}'_1 = \mathbf{Q}'_0$

$$P''(u) = \mathbf{P}_0 \cdot (12u - 6) + \mathbf{P}_1 \cdot (-12u + 6) + \mathbf{P}'_0 \cdot (6u - 4) + \mathbf{P}'_1 \cdot (6u - 2)$$

$$= \begin{pmatrix} u & 1 \end{pmatrix} \begin{bmatrix} 12 & -12 & 6 & 6 \\ -6 & 6 & -4 & -2 \end{bmatrix} \begin{bmatrix} \mathbf{P}_0 \\ \mathbf{P}_1 \\ \mathbf{P}'_0 \\ \mathbf{P}'_1 \end{bmatrix}$$

同理可得 $Q''(u)$

二阶导数连续的条件是 $P''(1) = a \cdot Q''(0)$ ，(a 是比例系数)，即

$$6\mathbf{P}_0 - 6\mathbf{P}_1 + 2\mathbf{P}'_0 + 4\mathbf{P}'_1 = -6\mathbf{Q}_0 + 6\mathbf{Q}_1 - 4\mathbf{Q}'_0 - 2\mathbf{Q}'_1,$$

$$\text{即 } 3\mathbf{P}_0 - 3\mathbf{P}_1 + \mathbf{P}'_0 + 2\mathbf{P}'_1 = -3\mathbf{Q}_0 + 3\mathbf{Q}_1 - 2\mathbf{Q}'_0 - \mathbf{Q}'_1,$$

$$\text{若 } \mathbf{P}_1 = \mathbf{Q}_0, \text{ 则 } 3\mathbf{P}_0 + \mathbf{P}'_0 + 2\mathbf{P}'_1 = 3\mathbf{Q}_1 - 2\mathbf{Q}'_0 - \mathbf{Q}'_1,$$

$$\text{若 } \mathbf{P}'_1 = a \cdot \mathbf{Q}'_0, \text{ 则 } 2(a+1)\mathbf{Q}'_0 = 3\mathbf{Q}_1 - \mathbf{Q}'_1 - 3\mathbf{P}_0 - \mathbf{P}'_0$$

两段 Hermite 形式的参数三次曲线光滑地拼接起来应满足

$$\begin{cases} \mathbf{P}_1 = \mathbf{Q}_0 \\ \mathbf{P}'_1 = a\mathbf{Q}'_0 \end{cases}$$